Raspberry Pi 5 – IST-Dokumentation des Kiosk-Systems

Projekt: Interaktive 3D-Tour „Haus und Garten“
Dokumentationsstand: 13. Oktober 2026 – periodischer Inaktivitäts-Neustart, Netzwerk-Wiederherstellung
Hardware: Raspberry Pi 5 mit 8 GB RAM
Betriebssystem: Raspberry Pi OS Lite, 64 Bit / Debian 13 (Trixie), ARM64 (`aarch64`)
Systembenutzer: `soundlux`
Startadresse: `https://storage.net-fs.com/hosting/7870141/23/#media=27`

> **Geltungsbereich:** Diese Dokumentation beschreibt die zuletzt gemeinsam eingerichtete **Betriebskonfiguration**. Die neue zyklische Timer-Logik ist nach den zuletzt besprochenen Änderungen dokumentiert; ihre wiederholte Auslösung ohne neue Eingabe wurde noch nicht durch ein Journalprotokoll bestätigt. Dateiinhalte sind kein direkter Export des Raspberry Pi. Ehemalige Versuche, deaktivierte Mechanismen und nicht genutzte Alternativen sind nicht Bestandteil der beschriebenen Architektur.

1. Systemübersicht

Nach dem Einschalten wird `soundlux` lokal auf `tty1` automatisch angemeldet. `~/.profile` startet die Wayland-Sitzung mit `labwc`. Über dessen Autostart läuft `~/kiosk/start-chromium.sh`; das Skript öffnet Chromium mit der 3D-Tour im Vollbild und startet den Browser nach Prozessende erneut.

Unabhängig davon läuft `/home/soundlux/kiosk/idle-watchdog.py` als aktivierter **systemd-Systemdienst**. Das Python-Skript liest über `evdev` tatsächliche Maus- und Touch-Eingaben. Nach Ablauf des konfigurierten Inaktivitätsintervalls (zuletzt **120 Sekunden** als aktiver Wert nachgewiesen) prüft es die Erreichbarkeit des Tour-Servers. Ist dieser erreichbar, beendet es Chromium per `pkill -TERM`; die bereits laufende Browser-Startschleife startet anschließend einen neuen Browserprozess mit der definierten Startadresse. **Der Timer beginnt nach jedem Timeout erneut – auch wenn keine weitere Benutzereingabe erfolgt.** Jede Maus- oder Touch-Eingabe setzt den Countdown weiterhin zurück.

Zusätzlich überprüft der Watchdog die HTTPS-Erreichbarkeit der Tour **alle 30 Sekunden**, bei erkanntem Netzwerkausfall **alle 15 Sekunden**. Bei fehlender Erreichbarkeit wird ein durch Inaktivität geplanter Browser-Neustart übersprungen; auch dann beginnt ein neues Inaktivitätsintervall. Sobald die Tour nach einem erkannten Ausfall wieder erreichbar ist, wird ein Chromium-Neustart angefordert und der Timer zurückgesetzt.

Die lokale Chromium-Erweiterung verhindert zusätzliche Tabs. Administrative Chromium-Richtlinien sperren externe Navigationen, Datei- und Speicherdialoge sowie Downloads.

```
RASPBERRY PI 5 — AKTIVER SYSTEMSTART
│
├── Raspberry Pi OS Lite · Debian 13 · ARM64
│   ├── LAN: eth0  /  Administration: SSH
│   ├── Autologin: soundlux auf tty1
│   │   └── /home/soundlux/.profile
│   │       └── dbus-run-session -- labwc
│   │           └── ~/.config/labwc/autostart
│   │               └── ~/kiosk/start-chromium.sh
│   │                   └── Chromium (Wayland, Kiosk-Vollbild)
│   │                       ├── 3D-Tour: .../#media=27
│   │                       ├── kiosk.json (Chromium-Policies)
```

```
│   │                       └── kiosk-tabblock/ (Tab-Sperre)
│   │
│   └── systemd: kiosk-idle-watchdog.service
│       └── ~/kiosk/idle-watchdog.py (Python + evdev)
│           ├── Eingaben erfassen: Maus / später Touch
│           ├── HTTPS-Erreichbarkeit der Tour prüfen
│           ├── TIMEOUT (zuletzt 120 s) ohne Eingabe + Tour online
│           │   └── pkill -TERM -u soundlux -x chromium
│           │       └── start-chromium.sh startet Chromium neu
│           ├── Timer nach jedem Intervall neu starten
│           │   └── auch ohne neue Eingabe: wiederholte Neustarts
│           ├── Tour offline: Browser geöffnet lassen
│           └── Tour wieder online: Chromium-Neustart anfordern
│
└── Chromium-Prozess beendet:
    start-chromium.sh protokolliert das Ende und startet neu.
```

Aktive Softwarekomponenten

Komponente	Aufgabe im aktuellen System
Raspberry Pi OS Lite	Schlankes Betriebssystem ohne vollständigen Desktop
SSH	Fernwartung über das Netzwerk
`labwc` / Wayland	Grafische lokale Sitzung
Chromium	Vollbildanzeige der online gehosteten 3D-Tour
Chromium Enterprise Policies	Sperre externer Navigationen, Downloads, Datei- und Speicherdialoge sowie Pop-ups
Chromium-Erweiterung (Manifest V3)	Neue Tabs verhindern bzw. direkt schließen
`python3-evdev`	Maus- und Touch-Eingaben aus `/dev/input/event*` lesen
`urllib.request` (Python-Standardbibliothek)	HTTPS-Erreichbarkeit des Tour-Servers prüfen
`pkill`	Chromium auf Anforderung des Watchdogs beenden
`start-chromium.sh`	Chromium nach Prozessende automatisch neu starten
`systemd`	Python-Watchdog automatisch starten und überwachen

Bei der Installation wurden `labwc 0.20.2` und `Chromium 154.0.8037.92` ausgegeben.

2. Aktive Dateistruktur

Im folgenden Baum stehen die für den aktuellen Betrieb relevanten Konfigurationsdateien, Skripte und vom System erzeugten Laufzeitdateien. Die eingerückten Einträge beschreiben **nicht** sämtliche Dateien des Raspberry Pi.

```
/
├── etc/
│   ├── chromium/
│   │   └── policies/
│   │       └── managed/
│   │           └── kiosk.json                  Chromium-Richtlinien
```

```
│   └── systemd/
│       └── system/
│           └── kiosk-idle-watchdog.service     Python-Systemdienst
│
├── home/
│   └── soundlux/
│       ├── .profile                            Start von labwc auf tty1
│       ├── .config/
│       │   ├── labwc/
│       │   │   └── autostart                    Chromium-Startschleife starten
│       │   └── chromium-kiosk/                  Browserprofil (von Chromium gepflegt)
│       ├── kiosk/
│       │   ├── start-chromium.sh                Browser-Neustartschleife
│       │   └── idle-watchdog.py                 Eingaben + Netzwerk + Neustart
│       ├── kiosk-tabblock/
│       │   ├── manifest.json                    Erweiterungsmanifest
│       │   ├── block.js                         Neue Fenster direkt abfangen
│       │   └── background.js                    Zusätzliche Tabs schließen
│       └── kiosk-chromium.log                   Log der Browser-Prozessenden
│
└── run/
    └── user/
        └── 1000/
            └── wayland-0                        Runtime-Socket der Wayland-Sitzung
```

`/run/user/1000/wayland-0` wird vom System zur Laufzeit erzeugt. `~/.config/chromium-kiosk/` wird von Chromium verwaltet und `~/kiosk-chromium.log` vom Browser-Startskript beschrieben. Das systemd-Journal speichert die Ausgaben des Python-Watchdogs zentral; es gibt dafür keine separate Logdatei im Home-Verzeichnis.

3. Lokale grafische Sitzung – `/home/soundlux/.profile`

Der Benutzer `soundlux` ist für den automatischen **Konsolen-Login auf `tty1`** konfiguriert. In `~/.profile` steht der Start der grafischen Sitzung:

```
# labwc nur an der lokalen Konsole starten
if [ "$(tty)" = "/dev/tty1" ] && [ -z "$WAYLAND_DISPLAY" ]; then
    exec dbus-run-session -- labwc
fi
```

- `$(tty)` identifiziert das angemeldete Terminal.
- `/dev/tty1` stellt sicher, dass labwc lokal auf dem Pi startet und **nicht bei SSH-Logins**.
- `[ -z "$WAYLAND_DISPLAY" ]` prüft, ob noch keine Wayland-Sitzung vorhanden ist.
- `dbus-run-session -- labwc` startet labwc mit einer D-Bus-Sitzung.
- `exec` ersetzt die Anmeldeshell durch diesen Prozess.

Der automatische Konsolen-Login wurde über `raspi-config` aktiviert.

4. labwc-Autostart – `/home/soundlux/.config/labwc/autostart`

Der derzeit verwendete Autostart startet **nur den Chromium-Watchdog**. Die Inaktivitätsüberwachung läuft separat als systemd-Dienst.

```
# Chromium mit automatischem Neustart
"$HOME/kiosk/start-chromium.sh" &
```

- `$HOME` ergibt für den lokalen Benutzer `/home/soundlux`.
- `start-chromium.sh` startet Chromium und überwacht sein Prozessende.
- `&` startet das Skript im Hintergrund, damit labwc nicht auf dessen Ende warten muss.

Ein Neustart des Pi lädt diese labwc-Konfiguration neu.

5. Chromium-Startskript – `/home/soundlux/kiosk/start-chromium.sh`

```
#!/bin/bash

URL="https://storage.net-fs.com/hosting/7870141/23/#media=27"

# Beim Systemstart kurz warten
sleep 3

# Chromium bei Beendigung automatisch neu starten
while true; do

    /usr/bin/chromium \
        --ozone-platform=wayland \
        --kiosk \
        --no-first-run \
        --no-default-browser-check \
        --password-store=basic \
        --noerrdialogs \
        --disable-session-crashed-bubble \
        --user-data-dir="$HOME/.config/chromium-kiosk" \
        --load-extension="$HOME/kiosk-tabblock" \
        "$URL"

    STATUS=$?

    echo "$(date -Is) Chromium beendet (Exit-Code: $STATUS)" \
        >> "$HOME/kiosk-chromium.log"

    sleep 3

done
```

Erklärung

Teil	Wirkung
`#!/bin/bash`	Verwendet Bash zur Skriptausführung.
`URL=...#media=27`	Legt die feste Startadresse der Tour fest.
`sleep 3` am Anfang	Wartet nach dem Start der grafischen Sitzung.
`while true ... done`	Startet Chromium nach jedem Prozessende erneut.

Teil	Wirkung
`--ozone-platform=wayland`	Verwendet Wayland als Grafik-Backend.
`--kiosk`	Öffnet die Tour im Kiosk-Vollbild.
`--no-first-run`	Unterdrückt die Ersteinrichtung.
`--no-default-browser-check`	Unterdrückt die Standardbrowser-Abfrage.
`--password-store=basic`	Verwendet die einfache Passwortspeicherung ohne Desktop-Keyring.
`--noerrdialogs`	Soll bestimmte Browser-Fehlerdialoge verhindern.
`--disable-session-crashed-bubble`	Soll die Absturz-Wiederherstellungs-Einblendung unterdrücken.
`--user-data-dir=...`	Nutzt das separate Profil `~/.config/chromium-kiosk/`.
`--load-extension=...`	Bindet die lokal erstellte Tab-Sperre ein.
`STATUS=$?`	Ermittelt den Exit-Code des Browser-Prozesses.
`echo ... >> kiosk-chromium.log`	Protokolliert Zeitpunkt und Exit-Code.
`sleep 3` am Ende	Wartet vor einem automatischen Neustart.

Bestätigter Test: Chromium wurde über SSH beendet und die Tour erschien wieder automatisch. Die Schleife überwacht **Prozessende**, nicht einen möglichen eingefrorenen, aber noch laufenden Browser.

6. Chromium-Richtlinien – `/etc/chromium/policies/managed/kiosk.json`

```
{
  "URLBlocklist": [
    "*"
  ],
  "URLAllowlist": [
    "storage.net-fs.com"
  ],
  "AllowFileSelectionDialogs": false,
  "DownloadRestrictions": 3,
  "DefaultFileSystemWriteGuardSetting": 2,
  "DefaultPopupsSetting": 2
}
```

Richtlinie	Aktive Funktion
`URLBlocklist: ["*"]`	Sperrt alle nicht freigegebenen Webseiten-Navigationen.
`URLAllowlist`	Erlaubt Navigation zur Tour-Domain.
`AllowFileSelectionDialogs: false`	Sperrt Dateiöffnen-/Speichern-Dialoge in Chromium.
`DownloadRestrictions: 3`	Sperrt Downloads.

Richtlinie	Aktive Funktion
`DefaultFileSystemWriteGuardSetting: 2`	Verweigert standardmäßig schreibende Website-Dateizugriffe über die betreffende API.
`DefaultPopupsSetting: 2`	Sperrt gewöhnliche JavaScript-Pop-ups.

Die Policies verhindern **das Laden** externer Seiten. Dass dabei keine zusätzlichen Tabs stehen bleiben, erreicht zusätzlich die lokale Erweiterung. Der Screenshot-Button der Tour führt noch seine eigene Animation aus; der Speicherdialog erscheint nicht mehr.

7. Chromium-Erweiterung – `/home/soundlux/kiosk-tabblock/`

Chromium lädt die Erweiterung über das Flag `--load-extension`. Der Quellcode der online gehosteten Tour wurde nicht verändert.

7.1 `manifest.json`

```
{
  "manifest_version": 3,
  "name": "Kiosk Tab Blocker",
  "version": "1.0",
  "permissions": ["tabs"],
  "background": {
    "service_worker": "background.js"
  },
  "content_scripts": [
    {
      "matches": ["https://storage.net-fs.com/*"],
      "js": ["block.js"],
      "run_at": "document_start",
      "all_frames": true,
      "world": "MAIN"
    }
  ]
}
```

- `manifest_version: 3` definiert das Erweiterungsformat.
- `permissions: ["tabs"]` erlaubt das Abfragen und Entfernen von Browser-Tabs.
- `background.service_worker` registriert `background.js`.
- `content_scripts` bindet `block.js` auf der Tour-Domain ein.
- `document_start` und `all_frames` sorgen für frühe Ausführung in passenden Dokumenten/Frames.
- `world: MAIN` führt das Script im JavaScript-Kontext der Seite aus.

7.2 `block.js`

```
(() => {
  // Links mit target="_blank" blockieren
  document.addEventListener("click", (event) => {
    const link = event.target?.closest?.('a[target="_blank"]');

    if (link) {
      event.preventDefault();
      event.stopImmediatePropagation();
    }
  }, true);
```

```
  // JavaScript-Popups und neue Fenster blockieren
  window.open = () => null;
})();
```

Der Listener verhindert die Standardaktion von angeklickten Links mit `target="_blank"`. `stopImmediatePropagation()` hält weitere Listener für dieses Ereignis an. `window.open` wird überschrieben, sodass Website-Aufrufe dieser Funktion kein Fenster öffnen.

7.3 `background.js`

```
chrome.tabs.onCreated.addListener(async (newTab) => {
  try {
    const tabs = await chrome.tabs.query({});

    const tourIsOpen = tabs.some(tab =>
      tab.id !== newTab.id &&
      tab.url?.startsWith(
        "https://storage.net-fs.com/hosting/7870141/23/"
      )
    );

    if (tourIsOpen && newTab.id !== undefined) {
      await chrome.tabs.remove(newTab.id);
    }
  } catch (error) {
    console.error("Tab konnte nicht geschlossen werden:", error);
  }
});
```

Das Hintergrundskript überwacht neu erstellte Tabs. Existiert bereits ein Tab mit der Tour, entfernt es den zusätzlichen Tab. Zusammen mit `block.js` wurde die Sperre externer Links erfolgreich getestet.

8. Python-Eingabe- und Netzwerk-Watchdog – `/home/soundlux/kiosk/idle-watchdog.py`

Der Watchdog läuft im Hintergrund als `systemd`-Systemdienst. Er erkennt angeschlossene Mäuse und Touchscreens anhand ihrer Linux-Eingabefähigkeiten und überwacht die Erreichbarkeit des Servers `storage.net-fs.com`. Die Inaktivitätsentscheidung hängt nicht von der automatischen Drehung der 3D-Tour oder von Chromiums Idle-Erkennung ab.

Python-Code gemäß zuletzt vorgenommener Anpassung

Zeitwert: Im zuletzt ausdrücklich bestätigten Betriebszustand war `TIMEOUT = 120` eingestellt. Eine spätere Umstellung auf 240 Sekunden wurde besprochen, aber nicht als durchgeführt bestätigt; deshalb ist unten **120** dokumentiert. Den auf dem Gerät tatsächlich eingestellten Wert zeigt `grep '^TIMEOUT =' ~/kiosk/idle-watchdog.py`. Die neue Wiederholungslogik wurde zuletzt geändert; ein Lognachweis mehrerer aufeinanderfolgender Neustarts **ohne jede Eingabe** wurde noch nicht vorgelegt.

```
#!/usr/bin/env python3

import time
import select
import subprocess
import urllib.request

from evdev import InputDevice, list_devices, ecodes


# Einstellungen
```

```
TIMEOUT = 120
NETWORK_CHECK = 30
NETWORK_RETRY = 15

TOUR_URL = "https://storage.net-fs.com/hosting/7870141/23/"

devices = {}
last_activity = time.monotonic()
idle_triggered = False

network_online = None
last_network_check = 0


def is_input_device(device):
    """Erkennt Mäuse und Touchscreens."""

    if device.name.startswith("vc4-hdmi"):
        return False

    caps = device.capabilities()

    rel = caps.get(ecodes.EV_REL, [])
    abs_axes = caps.get(ecodes.EV_ABS, [])
    keys = caps.get(ecodes.EV_KEY, [])

    mouse = (
        ecodes.REL_X in rel
        or ecodes.REL_Y in rel
        or ecodes.BTN_LEFT in keys
    )

    touchscreen = (
        ecodes.BTN_TOUCH in keys
        or ecodes.ABS_MT_POSITION_X in abs_axes
        or ecodes.ABS_MT_POSITION_Y in abs_axes
    )

    return mouse or touchscreen


def tour_reachable():
    """Prüft, ob der Tour-Server antwortet."""

    try:
        request = urllib.request.Request(
            TOUR_URL,
            headers={"User-Agent": "Mozilla/5.0"}
        )

        with urllib.request.urlopen(
            request, timeout=5
        ) as response:
            return 200 <= response.status < 400

    except Exception:
        return False


def restart_chromium():
    """Beendet Chromium; das Startskript startet es erneut."""

    print("Starte Chromium-Neustart ...", flush=True)

    try:
        result = subprocess.run(
            [
```

```
                "/usr/bin/pkill",
                "-TERM",
                "-u", "soundlux",
                "-x", "chromium"
            ],
            check=False,
            timeout=10
        )

        if result.returncode == 0:
            print(
                "Chromium-Neustart angefordert",
                flush=True
            )
        else:
            print(
                "Kein Chromium-Prozess gefunden",
                flush=True
            )

    except (OSError, subprocess.TimeoutExpired) as error:
        print(f"Neustart fehlgeschlagen: {error}", flush=True)


print("Kiosk-Watchdog gestartet", flush=True)


while True:

    # Eingabegeräte automatisch erkennen
    available = set(list_devices())

    for path in list(devices):
        if path not in available:
            devices.pop(path).close()
            print(f"Entfernt: {path}", flush=True)

    for path in available - set(devices):
        try:
            device = InputDevice(path)

            if is_input_device(device):
                devices[path] = device
                print(
                    f"Überwache: {path} – {device.name}",
                    flush=True
                )
            else:
                device.close()

        except OSError:
            pass

    # Maus- und Touch-Eingaben erkennen
    if devices:
        try:
            ready, _, _ = select.select(
                list(devices.values()), [], [], 0.5
            )

            for device in ready:
                for event in device.read():
                    if event.type in (
                        ecodes.EV_KEY,
                        ecodes.EV_REL,
                        ecodes.EV_ABS
                    ):
```

```
                        last_activity = time.monotonic()

                        if idle_triggered:
                            print("Wieder aktiv", flush=True)

                        idle_triggered = False

            except OSError:
                pass
        else:
            time.sleep(0.5)

        now = time.monotonic()

        # Netzwerk regelmäßig kontrollieren
        interval = (
            NETWORK_RETRY
            if network_online is False
            else NETWORK_CHECK
        )

        if now - last_network_check >= interval:

            current_status = tour_reachable()

            if network_online is False and current_status:
                print(
                    "Tour-Server wieder erreichbar",
                    flush=True
                )
                restart_chromium()
                idle_triggered = True
                last_activity = time.monotonic()

            elif network_online is True and not current_status:
                print(
                    "Tour-Server nicht erreichbar",
                    flush=True
                )

            elif network_online is None:
                print(
                    "Tour-Server erreichbar"
                    if current_status
                    else "Tour-Server nicht erreichbar",
                    flush=True
                )

            network_online = current_status
            last_network_check = time.monotonic()

        # Inaktivität erkennen
        if time.monotonic() - last_activity >= TIMEOUT:
            print(
                f"{TIMEOUT} Sekunden ohne Eingabe!",
                flush=True
            )

            idle_triggered = True

            # Vor einem Neustart Erreichbarkeit frisch prüfen
            if tour_reachable():
                network_online = True
                restart_chromium()

            else:
                print(
```

```
                "Neustart übersprungen: Tour-Server offline",
                flush=True
            )
            network_online = False

        # Countdown für das nächste Intervall neu beginnen
        last_activity = time.monotonic()
        last_network_check = time.monotonic()
```

Wie der Watchdog arbeitet

1. **`TIMEOUT = 120`:** Der zuletzt bestätigte Wert beträgt 120 Sekunden. Beim ersten Start beginnt der Countdown direkt beim Start des Watchdogs. Jede Eingabe aktualisiert `last_activity`. Nach jedem abgelaufenen Intervall wird `last_activity` erneut auf die aktuelle monotone Zeit gesetzt; dadurch kann der Timeout **auch ohne neue Eingabe immer wieder** ausgelöst werden. Das Intervall bezieht sich auf den Watchdog, nicht auf das Ende des Seitenladevorgangs.
2. **`is_input_device()`:** Erkennt Mäuse und Touchscreens anhand ihrer Tasten-, Bewegungs- und Touch-Achsen. HDMI-Geräte mit Namen `vc4-hdmi...` werden ignoriert. Gerätenummern wie `event1` können wechseln.
3. **`select.select()` und `device.read()`:** Lesen Ereignisse aus `/dev/input/event*`. Bei `EV_KEY`, `EV_REL` oder `EV_ABS` wird `last_activity = time.monotonic()` gesetzt: Der Countdown startet damit nach **jeder** Benutzereingabe neu.
4. **`idle_triggered`:** Hat nach der Änderung nur noch eine Rolle für die **Journalmeldung `Wieder aktiv`** nach einer Inaktivitätsphase. Es blockiert keinen weiteren Timeout mehr: Die Bedingung lautet jetzt allein `if time.monotonic() - last_activity >= TIMEOUT:`. Der Name der Variablen wurde im Skript beibehalten.
5. **`tour_reachable()`:** Ruft die HTTPS-Basisadresse der Tour ohne URL-Fragment ab. HTTP-Status 200 bis 399 zählt als erreichbar, die Anfrage hat fünf Sekunden Timeout. Das prüft den Webserver, nicht jeden 3D-Bestandteil.
6. **`NETWORK_CHECK = 30`:** Prüft im Normalbetrieb ungefähr alle 30 Sekunden die Erreichbarkeit.
7. **`NETWORK_RETRY = 15`:** Prüft bei einem Ausfall ungefähr alle 15 Sekunden erneut. Die Laufzeit der HTTP-Abfrage kann den Zeitpunkt verschieben.
8. **`restart_chromium()`:** Sendet mit `pkill -TERM -u soundlux -x chromium` ein Beendigungssignal an Chromium. Das separate `start-chromium.sh` startet Chromium mit der Tour erneut.
9. **Offline beim Timeout:** Es wird kein Browser-Neustart angefordert, solange der Tour-Server unerreichbar ist. Der Countdown wird trotzdem für das folgende Intervall zurückgesetzt. Die Netzwerkprüfung läuft unabhängig davon weiter.
10. **Netzwerk wiederhergestellt:** Beim Übergang von offline zu online wird ein Chromium-Neustart angefordert und mit `last_activity = time.monotonic()` ein neuer Countdown begonnen. Dieser Neustart kann auch während einer Benutzungsphase auftreten.
11. **Diagnose:** `Kiosk-Watchdog gestartet`, `Wieder aktiv`, `<TIMEOUT> Sekunden ohne Eingabe!`, `Tour-Server nicht erreichbar`, `Tour-Server wieder erreichbar` und `Chromium-Neustart angefordert` stehen im systemd-Journal.

Bestätigtes Verhalten

- Die direkte Maus-Eingabeüberwachung und der vollständige Chromium-Neustart wurden mit einem verkürzten Timeout mehrfach getestet.
- Mit **120 Sekunden** wurden die Inaktivität und der Chromium-Neustart im Journal bestätigt. Die **neue zyklische Wiederholung ohne erneute Eingabe** wurde anschließend eingerichtet, ist aber anhand eines solchen Journaltests noch nicht bestätigt.

- Ein temporär unerreichbar gemachter Tour-Server wurde erkannt; ein Neustart bei Inaktivität wurde korrekt übersprungen.
- Nach Wiederherstellung der Server-Erreichbarkeit wurde Chromium automatisch neu gestartet; die Tour erschien wieder auf dem Bildschirm.

Technische Grenzen der aktiven Lösung: Der Watchdog prüft die HTTPS-Erreichbarkeit der Startadresse, nicht alle 3D-Ressourcen oder den sichtbaren Rendering-Zustand. Eine Meldung `Chromium-Neustart angefordert` beweist, dass das Beendigungssignal gesendet wurde, nicht dass die Seite bereits vollständig geladen ist. Ein vollständig eingefrorener Browser, der nicht auf `SIGTERM` reagiert, wird dadurch nicht garantiert wiederhergestellt. Eine Rückkehr der Server-Verbindung kann im aktuellen Skript auch während einer laufenden Benutzerinteraktion einen Neustart anfordern.

9. systemd-Dienst – `/etc/systemd/system/kiosk-idle-watchdog.service`

```
[Unit]
Description=Kiosk Input Idle Watchdog
After=systemd-user-sessions.service

[Service]
Type=simple

# Warten, bis Wayland verfügbar ist
ExecStartPre=/bin/sh -c 'until test -S /run/user/1000/wayland-0; do sleep 1; done'

# Python-Watchdog starten
ExecStart=/usr/bin/python3 -u /home/soundlux/kiosk/idle-watchdog.py

# Nach einem Absturz automatisch neu starten
Restart=always
RestartSec=3

TimeoutStartSec=0

[Install]
WantedBy=multi-user.target
```

Einstellung	Bedeutung
`After=systemd-user-sessions.service`	Startreihenfolge relativ zur Einrichtung der Benutzersitzungen.
`Type=simple`	systemd überwacht den direkt gestarteten Python-Prozess.
`ExecStartPre=... wayland-0`	Wartet, bis die grafische Wayland-Sitzung ihren Socket bereitstellt.
`ExecStart=... python3 -u`	Startet den Watchdog mit ungepufferter Protokollausgabe.
Kein `User=`	Der Systemdienst läuft in der gezeigten Konfiguration als Root.
`Restart=always`	Startet den Watchdog nach Prozessende erneut.
`RestartSec=3`	Wartet drei Sekunden vor dem Neustart.

Einstellung	Bedeutung
`TimeoutStartSec=0`	Kein Zeitlimit für die Startphase mit Socket-Wartebedingung.
`WantedBy=multi-user.target`	Verknüpft den aktivierten Dienst mit dem normalen Systemstart.

Der Dienst wurde mit `systemctl enable --now kiosk-idle-watchdog.service` aktiviert. Bei der Prüfung war er als **`enabled` und `active (running)`** sichtbar. Erfolgreiche Chromium-Neustartanforderungen wurden im systemd-Journal protokolliert.

10. Netzwerk, Anzeige und Betrieb

Eingabe, Netzwerk und Bildschirm

Der Raspberry Pi nutzt aktuell **Ethernet (`eth0`)**. WLAN (`wlan0`) ist erkannt, aber bislang nicht verbunden. Fernwartung erfolgt über SSH. Auf dem HDMI-Monitor läuft Chromium im Kiosk-Vollbild; die Tour wurde mit einer USB-Maus getestet. Der spätere Touchscreen ist im Eingabefilter vorgesehen, aber noch nicht am realen Display verifiziert.

Für den Bildschirm ergab die Prüfung `consoleblank = 0`. Im aktiven labwc-Autostart ist kein Befehl zur Monitorabschaltung enthalten. Ein möglicher eigener Energiesparmodus des Monitors ist davon unabhängig.

Betriebsverhalten

Situation	Aktuelle Reaktion
TIMEOUT ohne erfasste Eingabe, Server erreichbar	`pkill` beendet Chromium; `start-chromium.sh` startet Chromium neu; ein neuer Countdown beginnt
Browser-Hauptprozess wird beendet	Browser-Startschleife startet Chromium nach kurzer Pause wieder
TIMEOUT abgelaufen, Tour-Server nicht erreichbar	Chromium bleibt geöffnet; nächstes Intervall beginnt, Netzprüfung läuft weiter
Tour-Server fällt aus	Im Journal erscheint `Tour-Server nicht erreichbar`; erneute Prüfungen ca. alle 15 Sekunden
Tour-Server wird wieder erreichbar	Chromium-Neustart wird automatisch angefordert, unabhängig vom Idle-Timer
Ohne weitere Eingabe nach Neustart	Nach dem nächsten TIMEOUT folgt erneut eine Neustartanforderung (sofern der Server erreichbar ist)

Die automatische Rotation der Tour ist ein Verhalten der Webseite. Sie erzeugt für sich genommen keine durch `evdev` erfasste Maus- oder Touch-Eingabe und setzt den Watchdog daher nicht zurück. Die periodischen Neustarts unterbrechen diese Rotation jeweils kurz.

Wichtig zur Nachweisbarkeit: Der neue zyklische Modus ist der zuletzt beschriebene Codezustand. Vor der Veranstaltung sollte einmal geprüft werden, ob im Journal ohne neue Eingabe **zwei aufeinanderfolgende** `Chromium-Neustart angefordert`-Meldungen mit ungefähr einem TIMEOUT Abstand erscheinen.

11. Diagnose und administrative Befehle

Prozess- und Dienststatus

```
# Grafische Sitzung, Chromium und Startschleife prüfen
pgrep -af 'labwc|chromium|start-chromium.sh'

# Systemdienst und Autostart prüfen
systemctl status kiosk-idle-watchdog.service
systemctl is-enabled kiosk-idle-watchdog.service

# Aktive Timeout- und Netzwerkintervalle anzeigen
grep -E '^(TIMEOUT|NETWORK_CHECK|NETWORK_RETRY) =' ~/kiosk/idle-watchdog.py

# Aktuelle labwc-Startkonfiguration
cat ~/.config/labwc/autostart
```

Ereignisse und Browser-Wiederanlauf beobachten

```
# Watchdog live verfolgen (SSH-Terminal kann dafür offen bleiben)
sudo journalctl -u kiosk-idle-watchdog.service -f

# Letzte 50 Watchdog-Meldungen
sudo journalctl -u kiosk-idle-watchdog.service -n 50 --no-pager

# Chromium-Prozessenden dokumentiert durch die Startschleife
tail -n 20 ~/kiosk-chromium.log

# Mehrere aufeinanderfolgende Neustarts ohne Eingabe prüfen
sudo journalctl -u kiosk-idle-watchdog.service --since today --no-pager | grep -E 'Sekunden ohne
Eingabe|Chromium-Neustart angefordert'
```

Erreichbarkeit und Systemzustand

```
# HTTP-Erreichbarkeit der eigentlichen Tour testen
python3 - <<'PY'
import urllib.request
url = "https://storage.net-fs.com/hosting/7870141/23/"
req = urllib.request.Request(url, headers={"User-Agent": "Mozilla/5.0"})
with urllib.request.urlopen(req, timeout=8) as response:
    print(response.status)
PY

# Aktueller Netzwerkstatus
nmcli device status

# CPU-Temperatur
vcgencmd measure_temp

# JSON der aktiven Chromium-Richtlinien validieren
python3 -m json.tool /etc/chromium/policies/managed/kiosk.json
```

Konfigurationsänderungen übernehmen

Änderungen an `~/kiosk/idle-watchdog.py` werden nach einem Neustart des Dienstes aktiv:

```
python3 -m py_compile ~/kiosk/idle-watchdog.py
sudo systemctl restart kiosk-idle-watchdog.service
```

Nach einer Änderung der Datei `/etc/systemd/system/kiosk-idle-watchdog.service` muss `systemd` zunächst die Dienstdefinition neu laden:

```
sudo systemctl daemon-reload
sudo systemctl restart kiosk-idle-watchdog.service
```

Änderungen an `~/.profile`, `~/.config/labwc/autostart` und an der Chromium-Startkonfiguration werden beim nächsten Start der grafischen Sitzung berücksichtigt. Im Projekt wurde dafür ein Systemneustart verwendet:

```
sudo reboot
```

IST-Stand: Die Tour startet automatisch, Browser-Richtlinien und Tab-Blocker sind wirksam, der vollständige Chromium-Neustart nach Inaktivität funktioniert, und die serverbasierte Netzwerk-Wiederherstellung wurde erfolgreich simuliert. Die zuletzt eingebaute Wiederholung des Timeouts **ohne neue Eingabe** ist beschrieben, aber noch nicht durch einen mehrzyklischen Journaltest belegt. Der reale Touchscreen-Test steht noch aus.